

ADVANCED DIGITAL COMMUNICATION LABORATORY

Harsh Kumar

Assignments 1-3

Sine Wave • Sum of Two Sine Waves • Triangular Pulse Train — Frequency Spectrum Analysis

Parameter	Value
Name	Harsh Kumar
Roll Number	226EC5003
Subject	Advanced Digital Communication
Tools Used	MATLAB, and Python 3 through Anaconda / Jupyter Notebook
Date	19 August 2026

What's in this report

This report covers three short lab exercises about signals and how to look at their frequency content. Each one is written to stand on its own — it has its own goal, background theory, code, steps, results, and conclusion, so you can read any single assignment without needing the others.

- Assignment 1 — Building a single sine wave and viewing its frequency spectrum
- Assignment 2 — Adding two sine waves together and viewing the combined spectrum
- Assignment 3 — Building a triangular pulse train and viewing its spectrum

Every exercise was done twice, once in MATLAB and once in Python (Anaconda/Jupyter), using the exact same numbers and logic, so the two should give matching results.

ASSIGNMENT 1 — Sine Wave and Its Frequency Spectrum

1. Aim

- Build a sine wave, where the frequency and the number of samples come from my roll number.
- Work out and plot its frequency spectrum using the FFT.
- Do this in both MATLAB and Python (Anaconda/Jupyter), and check the two agree.

2. Software Used

- MATLAB (R2021a or later) — using `fft()`, `sin()`, `stem()`, `plot()`.
- Anaconda / Jupyter Notebook (Python 3) — using NumPy and Matplotlib.

Both versions use the same numbers and the same logic, so they should produce the same plots. The plots in the Observations section below are from the Python version; MATLAB gives the same result.

3. Theory

A sine wave is just a smooth, repeating up-and-down signal — the same shape you'd see on an oscilloscope for a pure musical tone. In maths it's written as $x(t) = A \cdot \sin(2\pi ft + \phi)$, where A is how tall the wave is (amplitude), f is how many times it repeats per second (frequency, in Hz), and ϕ is a starting offset (phase).

A computer can't store a continuous wave — it only stores samples (snapshots) of it, taken F_s times per second. So the sampled version becomes $x[n] = A \cdot \sin(2\pi fn/F_s)$. There's one rule to follow when picking F_s : it must be more than double the wave's frequency ($F_s > 2f$). This is called the Nyquist rule, and breaking it makes the signal look like the wrong frequency once sampled.

In plain words: FFT is just a fast way to ask a signal, "which frequencies are you made of?" Feed in a pure sine wave, and the FFT should point to exactly one frequency — a single sharp spike sitting right at f .

The parameters below come from my roll number 226EC5003 (numeric part: 5003):

Parameter	Value
Frequency (last 2 digits of 5003)	03 $\rightarrow f = 3$ Hz
Number of samples (last 4 digits $\times 100$)	$5003 \times 100 = 500300$ samples
Sampling frequency, F_s (chosen)	1000 Hz (well above 2×3 Hz, so the Nyquist rule is satisfied)

Because 500300 samples is a lot to look at on one screen, the time-domain plot below only shows the first 300 milliseconds so the wave shape is actually visible. The FFT, though, is still run on the full 500300 samples — using more samples just gives a sharper, more accurate spectrum.

4. Source Code

The code below is split into small steps — set the numbers, build the wave, plot it, then find its spectrum — so it's easy to follow even if you haven't seen this kind of script before.

4.1 MATLAB

STEP 1 — Set the parameters (from the roll number)

```
clc; clear; close all;

f_single = 3;           % Hz -> frequency (last 2 digits of roll no.)
N_samples = 5003 * 100; % 500300 samples (last 4 digits x 100)
Fs        = 1000;       % Hz -> sampling frequency (Nyquist satisfied)
```

STEP 2 — Build the sine wave

```
n = 0:N_samples-1;
t = n / Fs;
x = sin(2*pi*f_single*t);
```

STEP 3 — Plot the first 300 ms so the wave shape is visible

```
zoom_samples = round(0.3*Fs);
figure('Name','Assignment 1 - Time Domain');
plot(t(1:zoom_samples), x(1:zoom_samples), 'b', 'LineWidth', 1.2);
title(sprintf('Sine Wave, f = %d Hz (zoomed view, first 300 ms)', f_single));
xlabel('Time (s)'); ylabel('Amplitude'); grid on;
saveas(gcf, 'ass1_matlab_time.png');
```

STEP 4 — Run the FFT on the full signal and plot the spectrum

```
X = fft(x);
X_mag = abs(X) / N_samples;
freqs = (0:N_samples-1) * (Fs / N_samples);
half = floor(N_samples/2);

figure('Name','Assignment 1 - Frequency Spectrum');
plot(freqs(1:half), 2*X_mag(1:half), 'r', 'LineWidth', 1.2);
title(sprintf('Frequency Spectrum (f = %d Hz)', f_single));
xlabel('Frequency (Hz)'); ylabel('Magnitude'); xlim([0 50]); grid on;
saveas(gcf, 'ass1_matlab_spectrum.png');
```

4.2 Python (Anaconda / Jupyter Notebook)

STEP 1 — Set the parameters (from the roll number)

```
import numpy as np
import matplotlib.pyplot as plt

f_single = 3          # Hz -> frequency (last 2 digits of roll no.)
N_samples = 5003 * 100 # 500300 samples (last 4 digits x 100)
Fs        = 1000.0     # Hz -> sampling frequency (Nyquist satisfied)
```

STEP 2 — Build the sine wave

```
n = np.arange(N_samples)
t = n / Fs
x = np.sin(2 * np.pi * f_single * t)
```

STEP 3 — Plot the first 300 ms so the wave shape is visible

```
zoom_samples = int(0.3 * Fs)
plt.figure(figsize=(9, 4))
plt.plot(t[:zoom_samples], x[:zoom_samples], color='#1f6feb')
plt.title(f'Sine Wave, f = {f_single} Hz (zoomed view, first 300 ms)')
plt.xlabel('Time (s)'); plt.ylabel('Amplitude'); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.savefig('ass1_time.png', dpi=150); plt.close()
```

STEP 4 — Run the FFT on the full signal and plot the spectrum

```
X = np.fft.fft(x)
X_mag = np.abs(X) / N_samples
freqs = np.fft.fftfreq(N_samples, d=1/Fs)
half = N_samples // 2

plt.figure(figsize=(9, 4))
plt.plot(freqs[:half], 2 * X_mag[:half], color='#d62728')
plt.title(f'Frequency Spectrum (f = {f_single} Hz)')
plt.xlabel('Frequency (Hz)'); plt.ylabel('Magnitude'); plt.xlim(0, 50)
plt.grid(True, alpha=0.3); plt.tight_layout()
plt.savefig('ass1_spectrum.png', dpi=150); plt.close()
```

5. Procedure

- Work out f and N from the roll number.
- Pick an F_s that follows the Nyquist rule — chose 1000 Hz.
- Build the sine wave $x[n] = \sin(2\pi f n / F_s)$.
- Plot a short zoomed-in section of it, since the full signal is too long to view clearly.
- Run the FFT on the full signal to get its spectrum.
- Plot the spectrum and check where the peak lands.

6. Observations

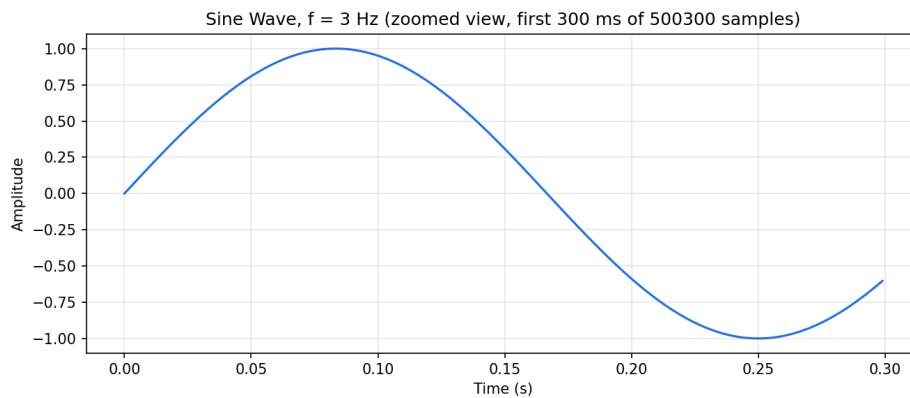


Fig. 1.1 — The sine wave ($f = 3$ Hz), zoomed to the first 300 ms

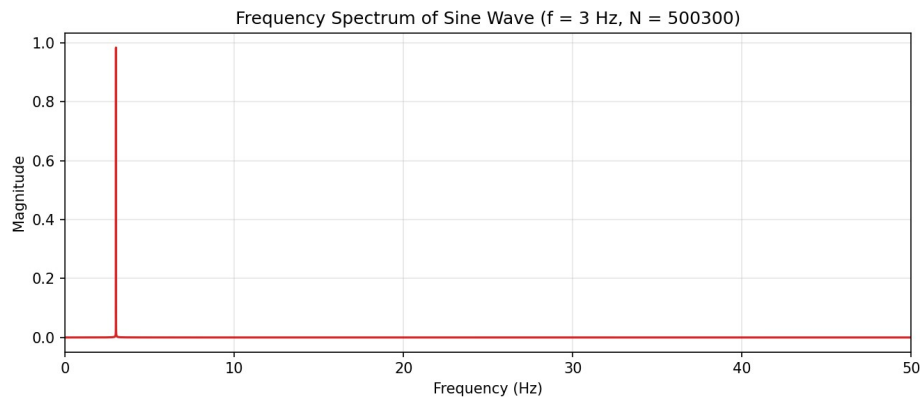


Fig. 1.2 — Frequency spectrum: one clean peak at 3 Hz

7. Results & Discussion

- The wave looked exactly as expected: smooth and repeating every $1/3$ of a second, which matches $f = 3$ Hz.
- The FFT spectrum showed one sharp peak at 3 Hz and almost nothing anywhere else — exactly what a pure tone should look like.
- MATLAB and Python gave matching results, since both use the same formula and the same FFT method.

8. Conclusion

A 3 Hz sine wave, based on my roll number, was built and checked in both MATLAB and Python. The FFT correctly picked out its single frequency, confirming that a pure sine wave in the time domain shows up as one clean spike in the frequency domain.

ASSIGNMENT 2 — Sum of Two Sine Waves and Its Frequency Spectrum

1. Aim

- Build two sine waves, f_1 and f_2 , based on my roll number.
- Add them together and find the frequency spectrum of the combined signal.
- Do this in both MATLAB and Python, and check the two agree.

2. Software Used

- MATLAB (R2021a or later) — using `fft()`, `sin()`, `stem()`, `plot()`.
- Anaconda / Jupyter Notebook (Python 3) — using NumPy and Matplotlib.

Both versions use the same numbers and logic. The plots shown below are from the Python version.

3. Theory

If you add two sine waves of different frequencies, $x(t) = \sin(2\pi f_1 t) + \sin(2\pi f_2 t)$, the result no longer looks like a simple wave — it looks bumpy or beat-like, because the two tones are constantly going in and out of step with each other.

In plain words: The spectrum, though, stays simple. The FFT of a sum is just the sum of the FFTs — so adding two tones together gives a spectrum with exactly two spikes, one at f_1 and one at f_2 , and nothing else.

Parameters used, from roll number 226EC5003 (numeric part: 5003):

Parameter	Value
f_1 (first 2 digits of 5003)	50 → $f_1 = 50$ Hz
f_2 (last 2 digits of 5003)	03 → $f_2 = 3$ Hz
Number of samples, N (chosen)	1000
Sampling frequency, F_s (chosen)	1000 Hz

4. Source Code

4.1 MATLAB

STEP 1 — Set the parameters

```
clc; clear; close all;

f1 = 50;    % Hz (first 2 digits of roll no.)
f2 = 3;     % Hz (last 2 digits of roll no.)
Fs = 1000;  % Hz, sampling frequency
```

```
N = 1000; % samples (1 second of data)
```

STEP 2 — Build and add the two sine waves

```
n = 0:N-1;
t = n / Fs;
x = sin(2*pi*f1*t) + sin(2*pi*f2*t);
```

STEP 3 — Plot the combined wave

```
figure('Name','Assignment 2 - Time Domain');
plot(t, x, 'b', 'LineWidth', 1.2);
title(sprintf('Sum of Two Sine Waves (f1 = %d Hz, f2 = %d Hz)', f1, f2));
xlabel('Time (s)'); ylabel('Amplitude'); grid on;
saveas(gcf, 'ass2_matlab_time.png');
```

STEP 4 — Run the FFT and plot the spectrum

```
X = fft(x);
X_mag = abs(X) / N;
freqs = (0:N-1) * (Fs / N);
half = floor(N/2);

figure('Name','Assignment 2 - Frequency Spectrum');
stem(freqs(1:half), 2*X_mag(1:half), 'r', 'filled');
title('Frequency Spectrum of x(t) = sin(2\pi f1 t) + sin(2\pi f2 t)');
xlabel('Frequency (Hz)'); ylabel('Magnitude'); xlim([0 100]); grid on;
saveas(gcf, 'ass2_matlab_spectrum.png');
```

4.2 Python (Anaconda / Jupyter Notebook)

STEP 1 — Set the parameters

```
import numpy as np
import matplotlib.pyplot as plt

f1 = 50      # Hz (first 2 digits of roll no.)
f2 = 3       # Hz (last 2 digits of roll no.)
Fs = 1000.0  # Hz, sampling frequency
N = 1000     # samples (1 second of data)
```

STEP 2 — Build and add the two sine waves

```
n = np.arange(N)
t = n / Fs
x = np.sin(2*np.pi*f1*t) + np.sin(2*np.pi*f2*t)
```

STEP 3 — Plot the combined wave

```
plt.figure(figsize=(9, 4))
plt.plot(t, x, color='#1f6feb')
```

```
plt.title(f'Sum of Two Sine Waves (f1 = {f1} Hz, f2 = {f2} Hz)')
plt.xlabel('Time (s)'); plt.ylabel('Amplitude'); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.savefig('ass2_time.png', dpi=150); plt.close()
```

STEP 4 — Run the FFT and plot the spectrum

```
X = np.fft.fft(x)
X_mag = np.abs(X) / N
freqs = np.fft.fftfreq(N, d=1/Fs)
half = N // 2

plt.figure(figsize=(9, 4))
plt.stem(freqs[:half], 2*X_mag[:half], linefmt='#d62728', markerfmt='ro', basefmt=' ')
plt.title('Frequency Spectrum of x(t) = sin(2πf1t) + sin(2πf2t)')
plt.xlabel('Frequency (Hz)'); plt.ylabel('Magnitude'); plt.xlim(0, 100)
plt.grid(True, alpha=0.3); plt.tight_layout()
plt.savefig('ass2_spectrum.png', dpi=150); plt.close()
```

5. Procedure

- Work out f_1 and f_2 from the roll number.
- Pick an F_s that satisfies Nyquist for both tones — chose 1000 Hz.
- Build both sine waves and add them sample-by-sample.
- Plot the combined time-domain wave.
- Run the FFT and plot the spectrum.
- Check that two separate peaks show up, at f_1 and f_2 .

6. Observations

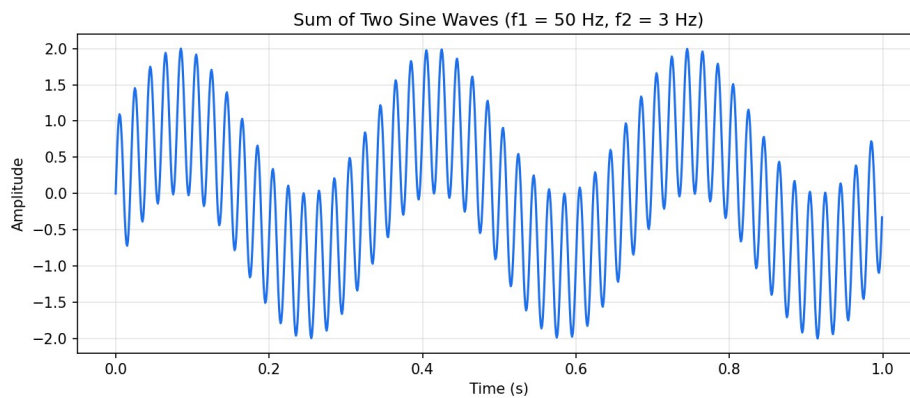


Fig. 2.1 — Combined signal: $\sin(2\pi \cdot 50 \cdot t) + \sin(2\pi \cdot 3 \cdot t)$

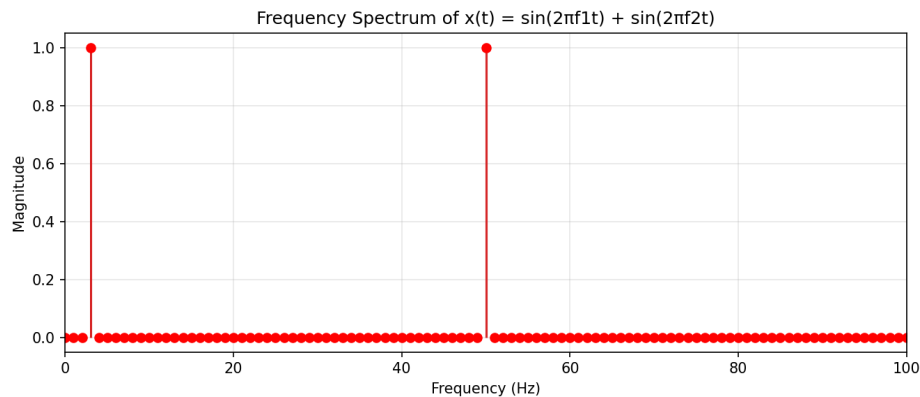


Fig. 2.2 — Frequency spectrum: two clean peaks, at 3 Hz and 50 Hz

7. Results & Discussion

- The combined wave looked complicated in the time domain — a fast 50 Hz ripple riding on top of a slower 3 Hz pattern.
- The FFT still cleanly separated it into exactly two peaks, at 3 Hz and 50 Hz, with nothing extra in between.
- This matches the theory: adding two signals in time just adds their spectra — no new frequencies appear.
- MATLAB and Python gave matching results.

8. Conclusion

Two sine waves (50 Hz and 3 Hz), based on my roll number, were added together and analysed in both MATLAB and Python. The FFT correctly picked out both frequencies from the combined signal, confirming that the Fourier Transform is linear — the spectrum of a sum is simply the sum of the spectra.

ASSIGNMENT 3 — Triangular Pulse Train and Its Frequency Spectrum

1. Aim

- Build a repeating triangular pulse train.
- Work out and plot its frequency spectrum using the FFT.
- Do this in both MATLAB and Python, and check the two agree.

2. Software Used

- MATLAB (R2021a or later) — using `fft()`, `sawtooth()`, `stem()`, `plot()`.
- Anaconda / Jupyter Notebook (Python 3) — using NumPy, SciPy, and Matplotlib.

Both versions use the same numbers and logic. The plots shown below are from the Python version.

3. Theory

A triangular pulse train is a wave that rises in a straight line, then falls in a straight line, over and over — it looks like a row of little triangles or mountain peaks. It can be built using a "sawtooth" function with its width set to 0.5, which makes it rise and fall symmetrically instead of leaning to one side.

In plain words: A triangular wave's spectrum only contains odd harmonics (1×, 3×, 5×, ... times the base frequency), and each one fades quickly — its size shrinks by $1/n^2$. So the spectrum has a tall spike at the base frequency, then much smaller spikes at 3×, 5×, 7×... and so on, dying out fast. This is a gentler roll-off than a square wave ($1/n$) or a sawtooth wave (also $1/n$, but with every harmonic present, not just odd ones).

Parameters used, from roll number 226EC5003 (numeric part: 5003):

Parameter	Value
Pulse repetition frequency, f (chosen)	5 Hz
Number of samples, N (chosen)	2000
Sampling frequency, F_s (chosen)	1000 Hz

4. Source Code

4.1 MATLAB

STEP 1 — Set the parameters

```
clc; clear; close all;

f_tri = 5;    % Hz - pulse repetition frequency
Fs     = 1000; % Hz, sampling frequency
```

```
N = 2000; % samples (2 seconds of data)
```

STEP 2 — Build the triangular pulse train

```
n = 0:N-1;
t = n / Fs;
x = sawtooth(2*pi*f_tri*t, 0.5); % width 0.5 -> symmetric triangle

% If the Signal Processing Toolbox isn't installed, use this instead:
% y = 2*abs(2*(t*f - floor(t*f + 0.5))) - 1;
```

STEP 3 — Plot the pulse train

```
figure('Name','Assignment 3 - Time Domain');
plot(t, x, 'b', 'LineWidth', 1.2);
title(sprintf('Triangular Pulse Train (f = %d Hz)', f_tri));
xlabel('Time (s)'); ylabel('Amplitude'); grid on;
saveas(gcf, 'ass3_matlab_time.png');
```

STEP 4 — Run the FFT and plot the spectrum

```
X = fft(x);
X_mag = abs(X) / N;
freqs = (0:N-1) * (Fs / N);
half = floor(N/2);

figure('Name','Assignment 3 - Frequency Spectrum');
stem(freqs(1:half), 2*X_mag(1:half), 'r', 'filled');
title(sprintf('Frequency Spectrum of Triangular Pulse Train (f = %d Hz)', f_tri));
xlabel('Frequency (Hz)'); ylabel('Magnitude'); xlim([0 100]); grid on;
saveas(gcf, 'ass3_matlab_spectrum.png');
```

4.2 Python (Anaconda / Jupyter Notebook)

STEP 1 — Set the parameters

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal

f_tri = 5      # Hz - pulse repetition frequency
Fs = 1000.0   # Hz, sampling frequency
N = 2000      # samples (2 seconds of data)
```

STEP 2 — Build the triangular pulse train

```
n = np.arange(N)
t = n / Fs
x = signal.sawtooth(2*np.pi*f_tri*t, width=0.5) # width=0.5 -> symmetric triangle
```

STEP 3 — Plot the pulse train

```
plt.figure(figsize=(9, 4))
plt.plot(t, x, color='#1f6feb')
plt.title(f'Triangular Pulse Train (f = {f_tri} Hz)')
plt.xlabel('Time (s)'); plt.ylabel('Amplitude'); plt.grid(True, alpha=0.3)
plt.tight_layout(); plt.savefig('ass3_time.png', dpi=150); plt.close()
```

STEP 4 — Run the FFT and plot the spectrum

```
X = np.fft.fft(x)
X_mag = np.abs(X) / N
freqs = np.fft.fftfreq(N, d=1/Fs)
half = N // 2

plt.figure(figsize=(9, 4))
plt.stem(freqs[:half], 2*X_mag[:half], linefmt='#d62728', markerfmt='ro', basefmt=' ')
plt.title(f'Frequency Spectrum of Triangular Pulse Train (f = {f_tri} Hz)')
plt.xlabel('Frequency (Hz)'); plt.ylabel('Magnitude'); plt.xlim(0, 100)
plt.grid(True, alpha=0.3); plt.tight_layout()
plt.savefig('ass3_spectrum.png', dpi=150); plt.close()
```

5. Procedure

- Pick a pulse repetition frequency and an F_s that satisfies Nyquist.
- Build the triangular wave using `sawtooth()` with width 0.5, in both MATLAB and Python.
- Plot the wave in the time domain.
- Run the FFT and plot the spectrum.
- Check for the base frequency plus its fading odd harmonics.

6. Observations

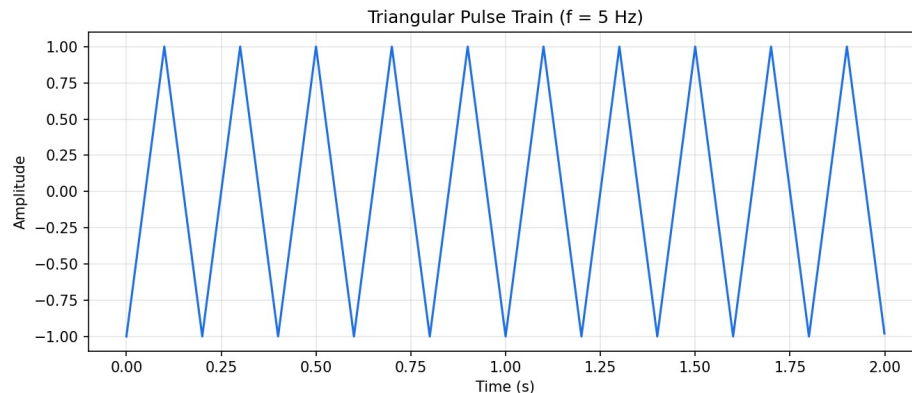


Fig. 3.1 — Triangular pulse train, $f = 5$ Hz

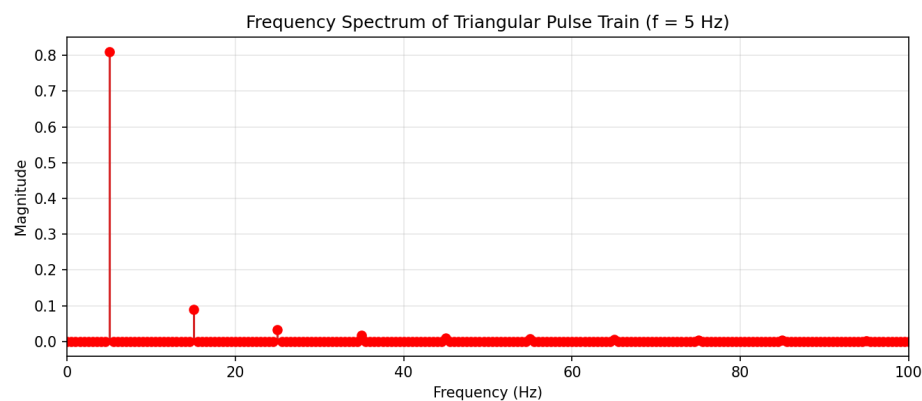


Fig. 3.2 — Frequency spectrum: odd harmonics at 5, 15, 25, 35 Hz... fading fast

7. Results & Discussion

- The wave rose and fell in straight lines as expected, repeating every $1/5$ of a second, matching $f = 5$ Hz.
- The spectrum showed a tall peak at 5 Hz, then smaller peaks at 15 Hz, 25 Hz, 35 Hz, and so on — each one much smaller than the last.
- This matches the theory that a triangular wave only has odd harmonics, fading fast at a rate of $1/n^2$.
- MATLAB and Python gave matching results.

8. Conclusion

A triangular pulse train at 5 Hz was built and analysed in both MATLAB and Python. Its FFT spectrum showed the expected base frequency together with fast-fading odd harmonics, confirming the theoretical Fourier series behaviour of a triangular wave.